

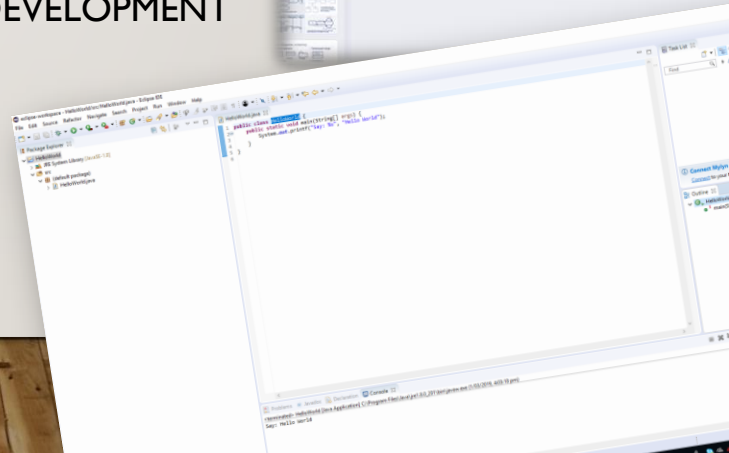


THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

MODEL-VIEW-CONTROLLER (MVC) PATTERN

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

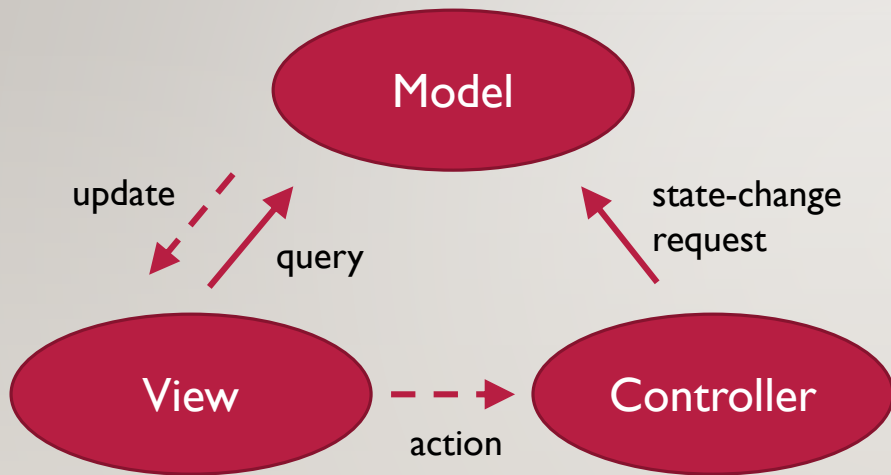
PARAMVIR SINGH



AGENDA

- The Model View Controller (MVC) pattern
- MVC and Swing
 - JTextComponent
- Swing Tables
- Code resources
 - Package musicApp
- *Important Reading Resources*

MODEL VIEW CONTROLLER (MVC)



The MVC pattern promotes a separation of concerns in terms of presentation, logic and data representation

- **Model**

- Encapsulates application state
- Responds to state queries
- Exposes functionality allowing its state to be updated
- Notifies registered views of changes in its state

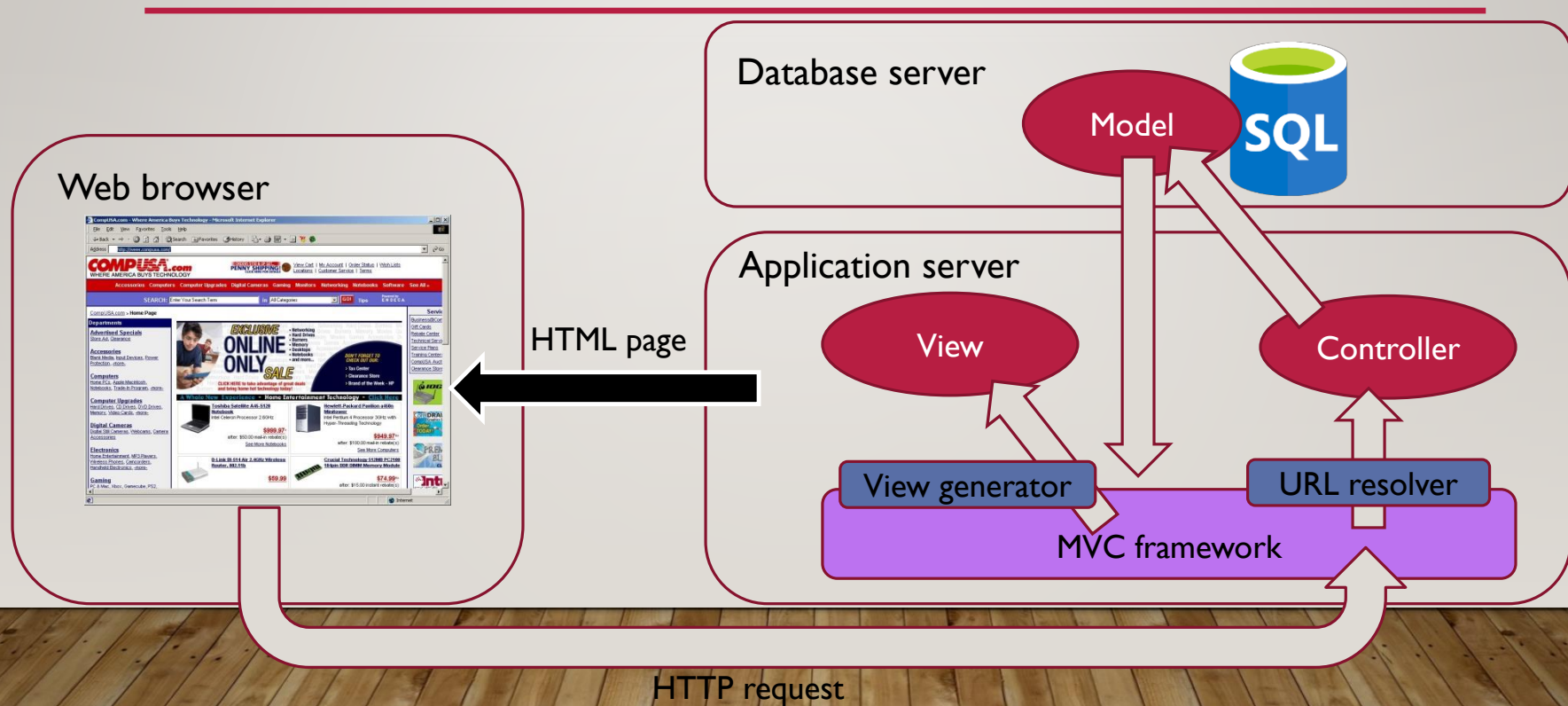
- **View**

- Renders the model
- Requests state data from the model
- Sends user actions to a registered controller

- **Controller**

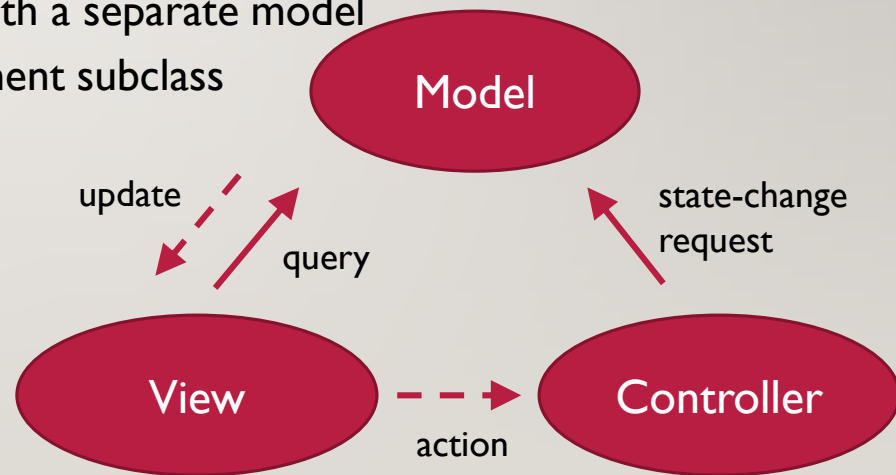
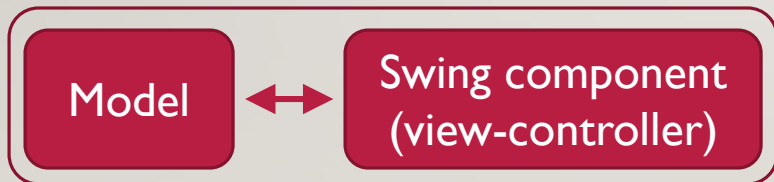
- Maps user actions onto model messages

MVC FOR WEB APPLICATIONS



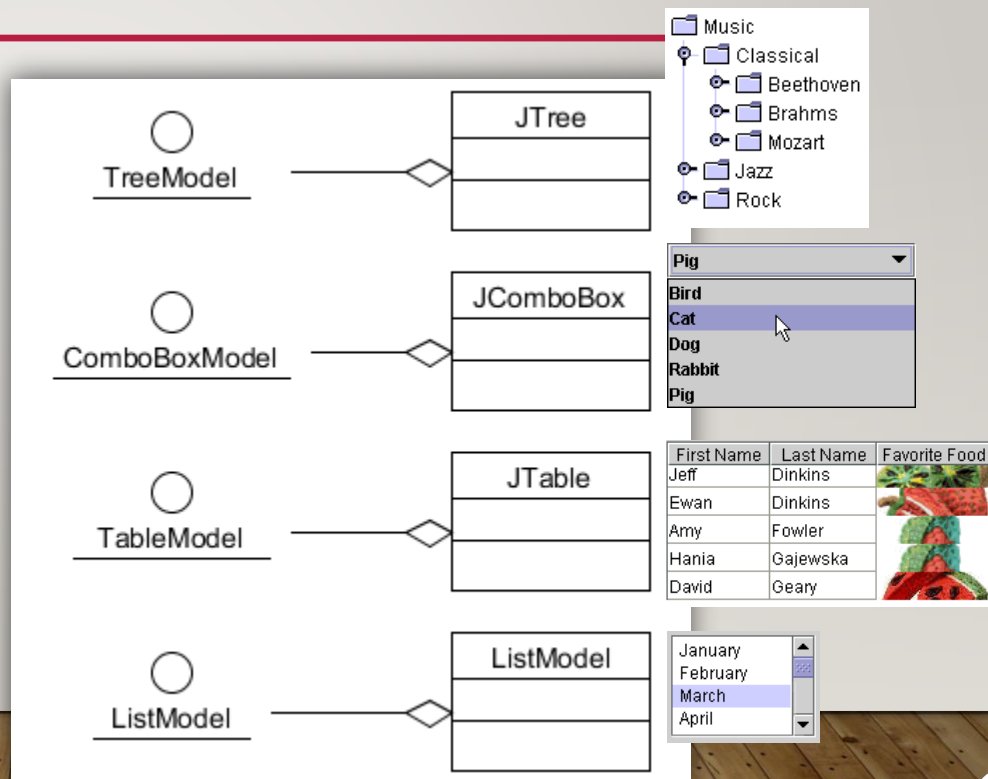
MVC AND SWING

- Swing components are rooted in MVC principles
 - A Swing component is a view-controller
 - A Swing component is associated with a separate model
 - A Swing component is any JComponent subclass



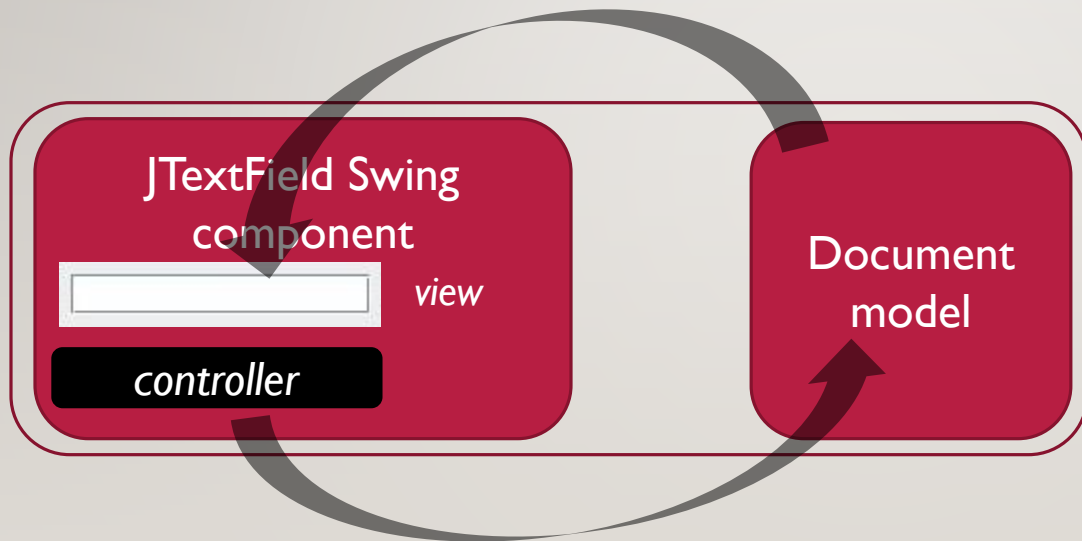
SWING COMPONENTS AND MODELS

- For each Swing component type, there is a corresponding model interface
- A JComponent will work with any object whose class implements its model interface
- The model interfaces differ – but the model/view principles are common



TEXT COMPONENTS IN SWING

2: insertUpdate()



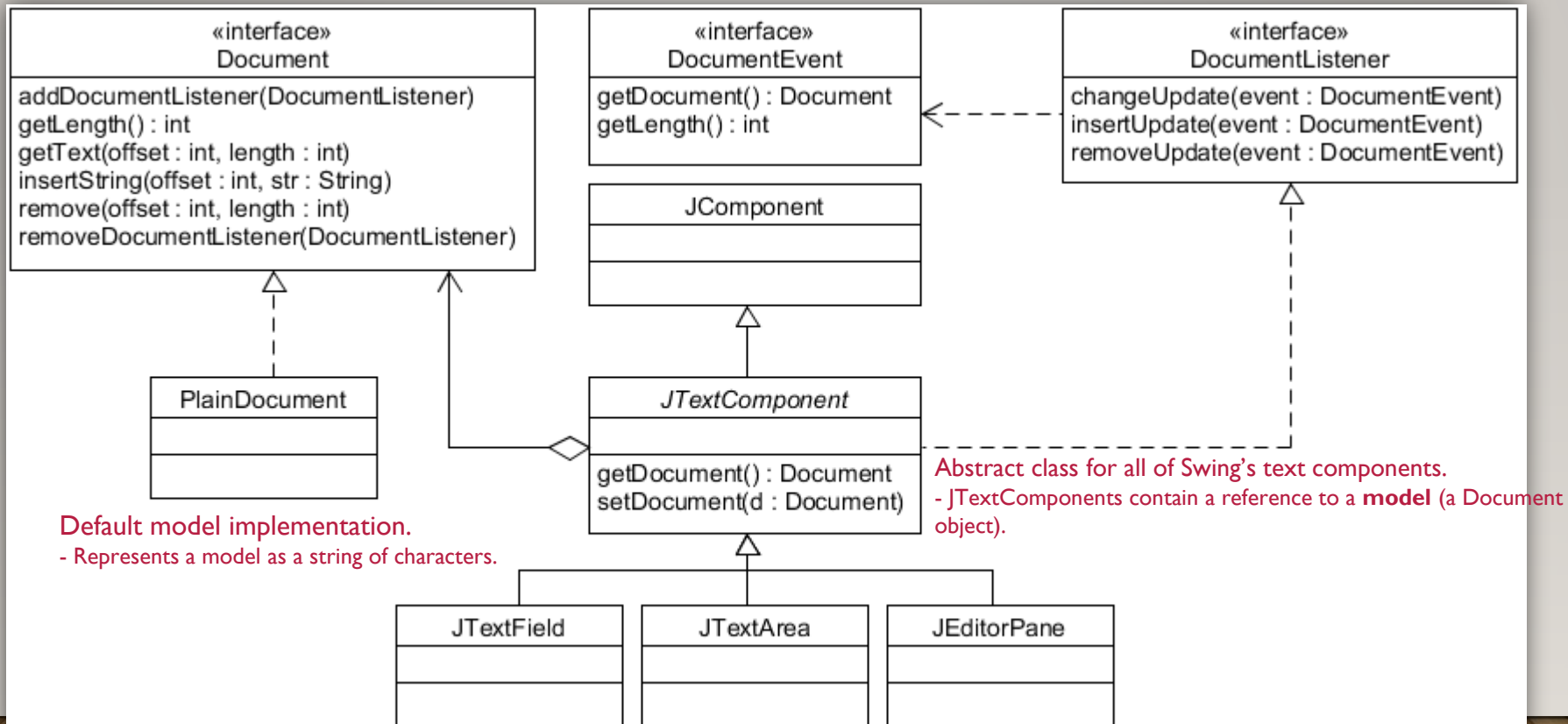
1: insertString()

- A **JTextField**'s controller handles user input and attempts to update its associated model
- Only if the model changes its state does it notify its view – and this is the point at which the view refreshes its display

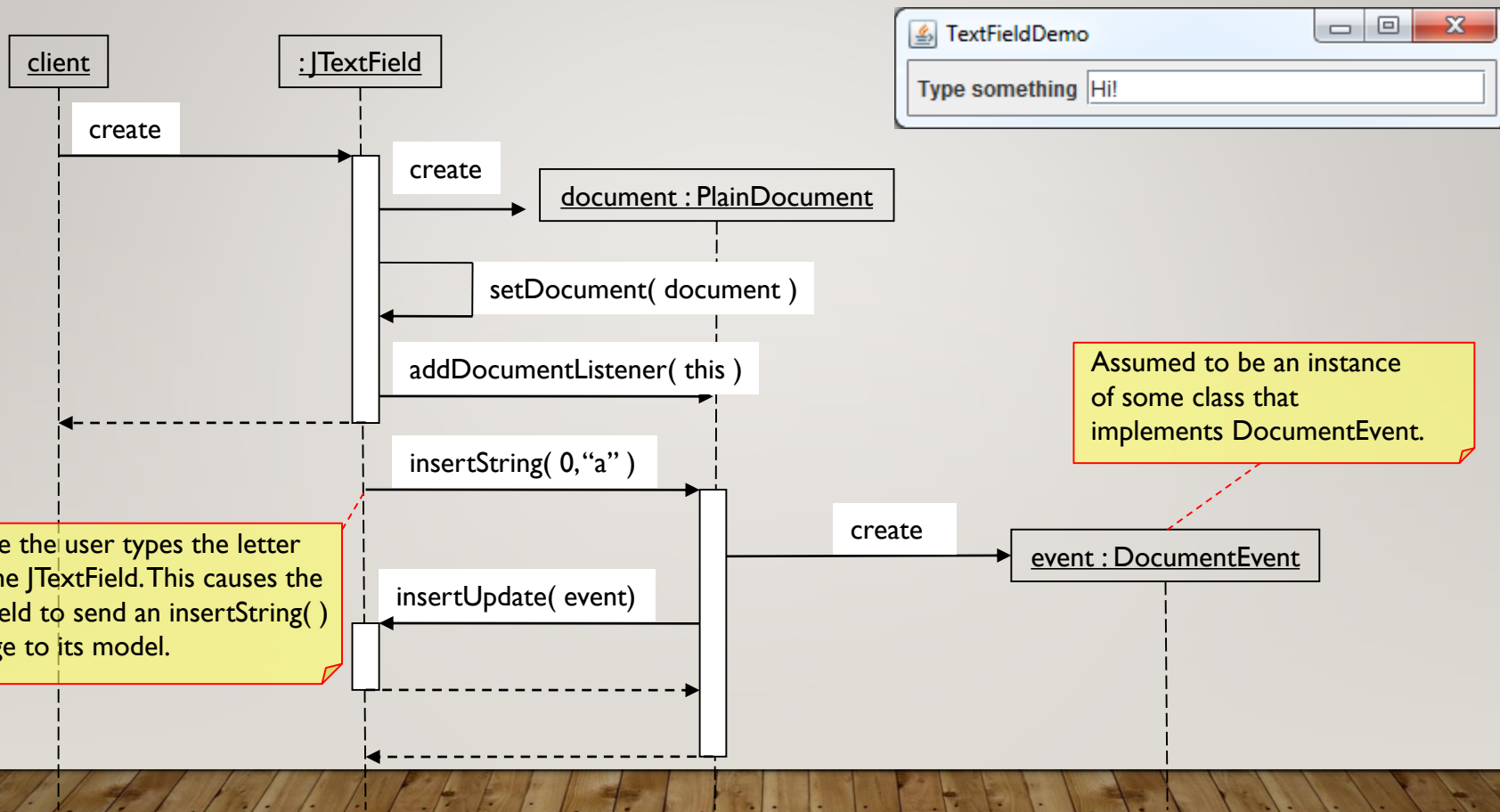
MVC CLASS INTERACTIONS IN JTEXTCOMPONENT

Model interface

View interface

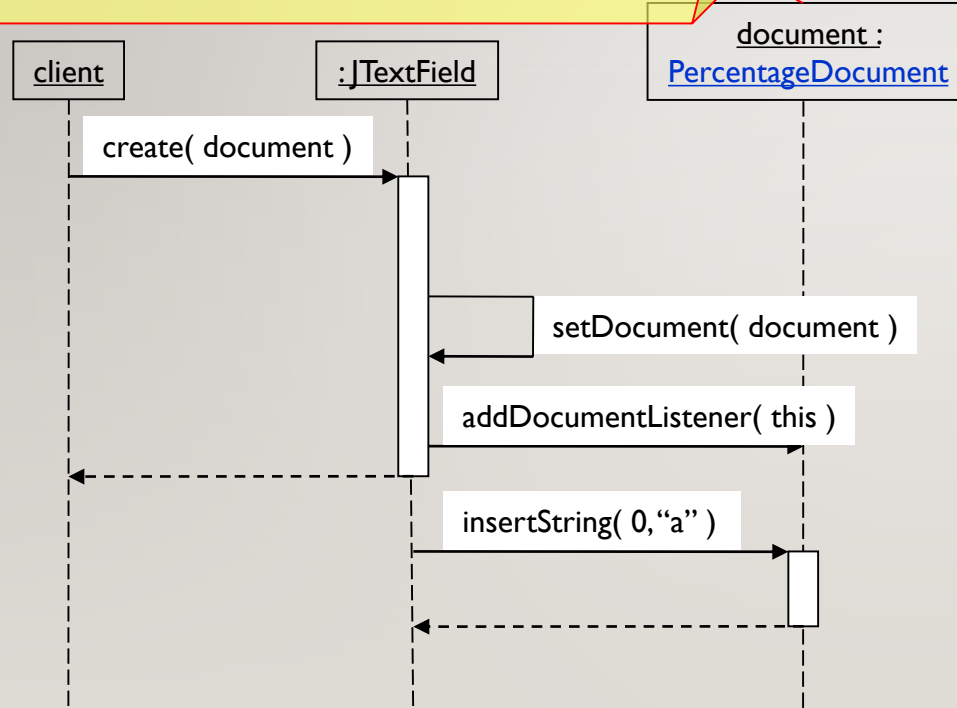


MVC CLASS INTERACTIONS IN JTEXTCOMPONENT



A Document implementation that constrains its character sequence as follows:

1. The length must be ≤ 3 .
2. Characters must be digits.
3. The sequence should include no leading zeros.
4. When converted to a number, it should be ≤ 100 .



```
public class TextFieldDemo extends JPanel {
```

```
    public TextFieldDemo() {
        JLabel label = new JLabel("Type something");
        JTextField textField = new JTextField(20);
```

```
        Document document = new PercentageDocument();
        textField.setDocument(document);
```

```
        add(label);
        add(textField);
    }
```

```
    private static void createAndShowGUI() {
        JFrame frame = new JFrame("TextFieldDemo");
```

```
        JComponent demo = new TextFieldDemo();
```

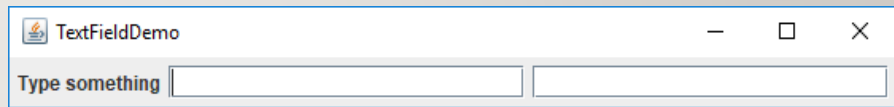
```
        frame.add(demo);
        frame.pack();
        frame.setVisible(true);
```

```
    }
```

```
    public static void main(String[] args) {
        createAndShowGUI();
    }
}
```

USING MVC TO CREATE MULTIVIEW EDITORS

- Swing's predecessor, AWT, isn't based on MVC principles
 - How would you develop a multiview text editor using the AWT?
- Note how the MVC-based scheme guarantees that all views are updated in response to a model change
 - This contributes to robustness – all views should always be mutually consistent
- Views needn't be visual



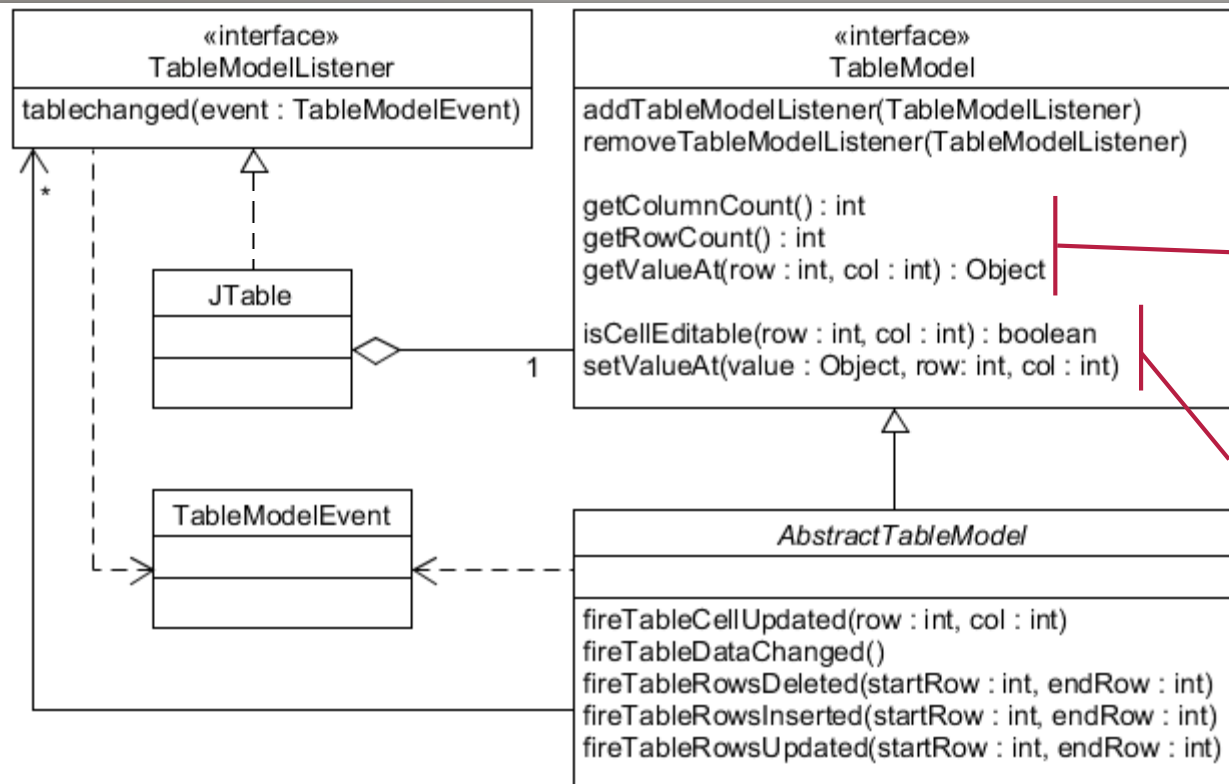
MVC AND SWING

MULTIVIEW TEXT EDITOR USING JTEXTAREA?

- How would you develop a multiview “text area” in Swing?
- Construct a class diagram to show the classes and their relationships required.
- Construct a sequence diagram to show what happens when text in one view is modified.

```
public class MultiViewTextEditor extends JPanel {  
  
    private Document _model;  
    private JTextArea _view1;  
    private JTextArea _view2;  
  
    public MultiViewTextEditor() {  
        _model = new PlainDocument();  
        _view1 = new JTextArea();  
        _view2 = new JTextArea();  
  
        _view1.setDocument(_model);  
        _view2.setDocument(_model);  
        ...  
    }  
    ...  
}
```

SWING TABLES: JTABLE



Code reuse methods, implemented in `AbstractTableModel`, to attach and remove views.

Concept reuse methods that must be implemented by `AbstractTableModel` subclasses for a `JTable` to extract sufficient data to build the visual representation of the model

Concept reuse methods with default implementations in `AbstractTableModel`. `AbstractTableModel` subclasses should override these methods to allow the model to be edited through the `JTable` view-controller.

Code reuse methods that are implemented and which provide a convenient facility for subclasses to fire `TableModelEvents`

SWING TABLES: JTABLE

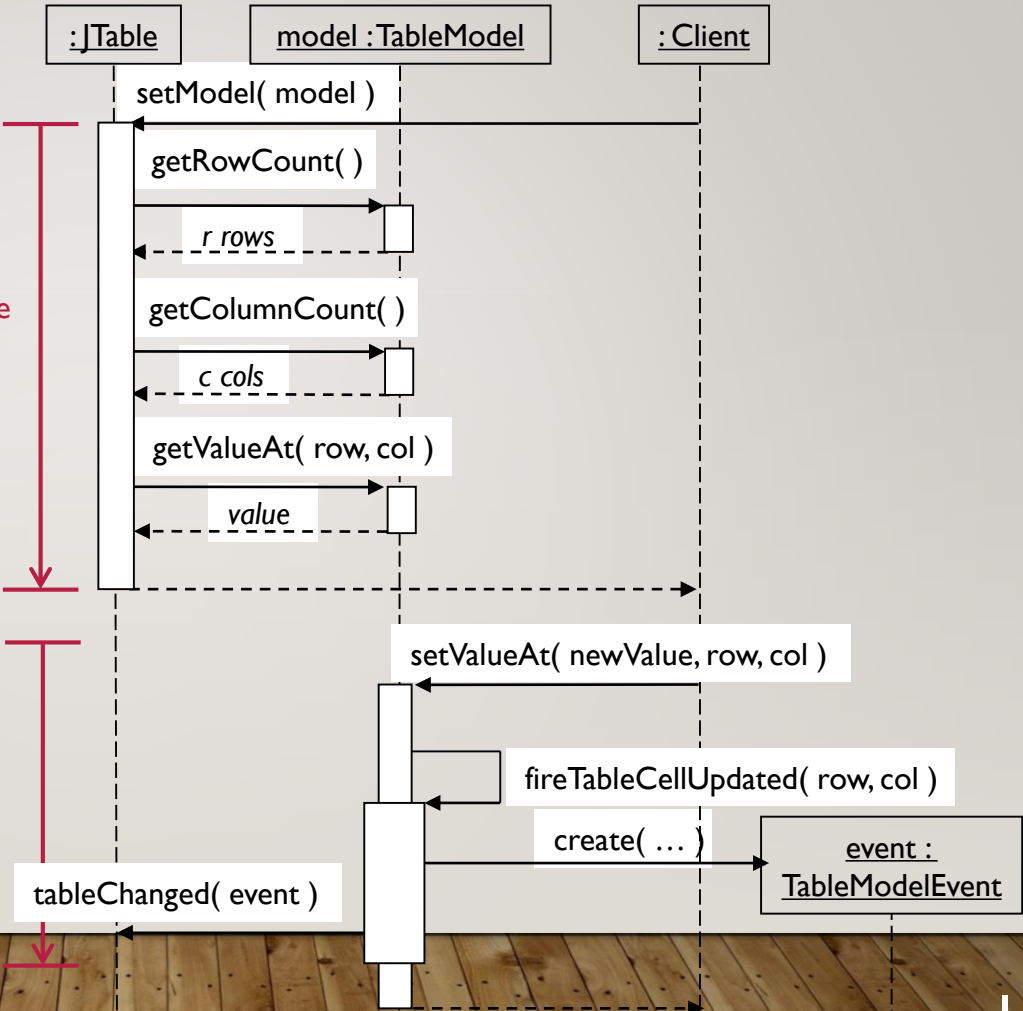
Setting up the model and view

When introduced to its model, the view (JTable) asks the model how many rows and columns it has. It then calls `getValueAt()` $r * c$ times, obtaining a value for each cell.

The view renders each value.

Handling a change request

In response to a change request on the model (i.e. a `setValueAt()` call), if the model accepts the change it creates a `TableModelEvent` object to describe the change, and sends this to its view(s).



SWING TABLES

- You want to develop a simple Swing application to include a **JTable** listing your favourite music tracks
- You already have class **MusicTrack** and a main program that populates a GUI with a **JTable**
- Identify and write the additional code that is necessary to populate the JTable with a set of music tracks

```
public class MusicTrackApp extends JFrame {  
  
    private JTable _tableComponent;  
  
    public MusicTrackApp() {  
        _tableComponent = new JTable();  
        add(_tableComponent);  
    }  
  
    public static void main(String[] args) {  
        MusicTrackerApp frame = new MusicTrackApp();  
        frame.pack();  
        frame.setVisible(true);  
    }  
}
```

```
public class MusicTrack {  
  
    private String _title;  
    private String _artist;  
    private String _album;  
  
    Getter and setter methods ...  
}
```

SUGGESTED TASKS

- Implement the **Multiview Text Editor** app
- Complete the **MusicTrack** app
 - Source code is supplied to represent music tracks (MusicTrack), to read music data from file (MusicTrackReader) and to create a skeletal GUI containing a JTable (MusicTrackApp)



The screenshot shows a Java Swing window titled "MusicTracker". Inside the window is a JTable with three columns: "Title", "Artist", and "Album". The table contains 25 rows of music track data. The window has standard Mac OS X window controls (red, yellow, green buttons) in the top right corner.

Title	Artist	Album
Don't Change	INXS	Shabooh Shoobah
Listen Like Thieves	INXS	Listen Like Thieves
Kiss The Dirt	INXS	Listen Like Thieves
Baby Don't Cry	INXS	Welcome to Wherever You...
New Sensation	INXS	Kick
Never Teat Us Apart	INXS	Kick
Mystify	INXS	Kick
Elegantly Wasted	INXS	Elegantly Wasted
Suicide Blonde	INXS	X
Bitter Tears	INXS	X
Beds Are Burning	Midnight Oil	Diesel and Dust
Blue Sky Mine	Midnight Oil	Blue Sky Mining
Put Down That Weapon	Midnight Oil	Diesel and Dust
The Dead Heart	Midnight Oil	Diesel and Dust
When the War is Over	Cold Chisel	Circus Animals
Forever Now	Cold Chisel	Circus Animals
Cheap Wine	Cold Chisel	East
You Got Nothing I Want	Cold Chisel	East
The Boys Light Up	Australian Crawl	The Boys Light Up
Down Under	Men at Work	Business as Usual
Who Can it Be Now?	Men at Work	Business as Usual

SUGGESTED TASKS

- Study the **JTable** and **JTree** Swing components from the following resources and work to make **TableModelAdapter** listen to the changes in **TreeModelAdapter** or vice-versa. That is, changes in one brings changes in the other.
- In our multiview editor, we had a single model working behind the two views. In this case, you would need two different models (or model adapters) controlling JTable view and JTree view, respectively.
- Study/Web links
 - <https://docs.oracle.com/javase/tutorial/uiswing/components/table.html>
 - <https://docs.oracle.com/javase/tutorial/uiswing/components/tree.html>
 - <https://www.informit.com/articles/article.aspx?p=332278&seqNum=2>
 - <https://www.codejava.net/java-se/swing/editable-jtable-example>
 - <https://www.consulting-bolte.de/index.php/11-tech-blog/java/java-se/40-java-filetreemodel>
 - <http://www.java2s.com/Code/Java/Swing-JFC/implementsTreeModeltocreatetreemodel.htm>
 - <http://www.java2s.com/Code/Java/Swing-JFC/Thisprogramdemonstrateshowtouseacustomtreemodelltdisplaysthefieldsofanobject.htm>

REVIEW

- Learnt something about:
 - Swing's separable model architecture
 - A model is responsible for data structures and logic
 - Components are view-controllers and concerned with the GUI
 - A component's controller handles user input
 - A view is part of a component and provides a (visual) representation of the model
 - Controllers make change requests of the model; the model may or may not accept the request
 - A model guarantees to notify its view following any state change in the model
 - A model can have multiple views to which the model broadcasts a notification following a state change

REFERENCES

- MVC and Swing:
 - <https://www.oracle.com/technical-resources/articles/javase/mvc.html>
 - <https://www.oracle.com/java/technologies/a-swing-architecture.html>
- More useful links:
 - <https://docs.oracle.com/javase/tutorial/uiswing/components/table.html>
 - <https://docs.oracle.com/javase/tutorial/uiswing/components/tree.html>
 - <https://www.informit.com/articles/article.aspx?p=332278&seqNum=2>
 - <https://www.codejava.net/java-se/swing/editable-jtable-example>
 - <https://www.consulting-bolte.de/index.php/I-I-tech-blog/java/java-se/40-java-filetreemodel>
 - <http://www.java2s.com/Code/Java/Swing-JFC/implementsTreeModeltocreatetreemodel.htm>
 - <http://www.java2s.com/Code/Java/Swing-JFC/Thisprogramdemonstrateshowtouseacustomtreemodelltdisplaysthefieldsofanobject.htm>